

CS 170 Lab 3: Debugging

When something is wrong in code, it is called a “bug,” and the process of finding and fixing the error is called “debugging.” While the exact origin of the term is unknown, the Smithsonian Museum in Washington D.C. does have on exhibit a moth whose presence in a 1947 computer at Harvard University caused computational issues (https://americanhistory.si.edu/collections/object/nmah_334663). Their research indicates the incident popularized the terms of bug and debugging in relation to computer programs.

Syntax Errors / Compile-Time Errors

- Examples: missing semicolons, misspelled keywords, uppercase instead of lowercase letters, mismatched brackets, etc.
- These errors are caught by the compiler (javac).
- They are the easiest to fix and come with a specific error message.

Logical Errors / Runtime Errors

- The program compiles (javac gives no error) but execution gives the wrong result.
- These errors can be trickier to detect and correct.
- It's very useful to trace the program, i.e., manually simulate the execution of the program without having the machine execute it for you. While you trace the program, compare what the program is doing with what it is supposed to do.
- The program may crash, have an infinite loop, or given an incorrect result.

Finding Logical Errors Through Testing

- Try all the examples that are given to you with the problem
- Think of edge cases or extreme cases: zero, smallest or biggest possible values, boundaries of conditional statements, beginning and ending values in loops, etc.
- It is helpful to write your test cases even before writing the code
 - It helps you understand the problem better
- Test your code often while you program
 - Incrementally test and fix partial programs

Debugging Tips

- Reproduce the problem
 - Find the input values that trigger the problem
 - What is the program supposed to do? (expected output)
 - What is the program doing instead? (real output)
- Examine the program behavior for that input
 - Code tracing
 - Visualize the internal state of the program with temporary print statements
- Fix only one thing at a time so you don't cascade errors
- Retest all your test cases: Your fix might have broken something else!
- Repeat the process until all bugs are fixed
- Clean up your code
 - Remove all temporary print statements, temporary variables, breaks, etc.

Below are 5 Java methods. Unfortunately, none of the methods work! Each method has **exactly 2** errors. For each method, (1) identify the two errors and what type of error they are (logical vs. syntax), and (2) fix the errors so the code is working. You may copy the code to a code editor to test the programs.

1. compareNumbers

```
/* The following method is supposed to return a textual
description of the comparison between two numbers (like "XXX is
smaller than YYY" etc.) */
```

```
1 public static String compareNumbers(int x, int y) {
2     if (x < y) {
3         return x + " is smaller than " + y;
4     } else if (x >= y) {
5         return x + " is greater than " + y;
6     } else {
7         return x + " is equal to " + y;
8     }
9 }
```

Error 1

Line Number	Error Type	Error Description	Fix
8	Syntax	No closing curly brace for the method	Add an additional curly brace at the end

Error 2

Line Number	Error Type	Error Description	Fix
4	Logical	Statement will be true if x is greater than or equal to y but we only want greater than	Remove the = from >=

2. meetings

/* The following method is supposed to take a number of students, a minimum length of a meeting, and a number of minutes available for student meetings, and return a message explaining the division of meeting time among students.

For example, for 2 students with minimum time 20 minutes and total minutes available 50, the method should return "Each student will have a 25 minute meeting with the professor, and the professor will have 0 minutes of extra time." */

Add. test cases: 3, 20, 50 2, 20, 30 2, 20, 40 2, 20, 55 -1, 2, 30
1, 2, 30

```

1 public static String meetings(int students, int minimum, int
  minutes) {
2     String answer = "";
3     if (students < 0 && minutes < 0) {
4         answer = "There's no such thing as negative students or
          negative minutes!";
5     } else if (students * minimum > minutes) {
6         answer = "There is not enough time for all student
          meetings. ";
7         answer += "Professor must find at least " +
          (students * minimum - minutes) + " more minutes!";
8     } else {
9         answer = "Each student will have a " +
          (minutes % students) + " minute meeting with the
          professor, " + "and the professor will have " +
          (minutes % students) + " minutes of extra time.";
10    }
11    return answer;
12 }

```

returns remainder
w/ 2 students
and 50 minutes
returns 0

Error 1

Line Number	Error Type	Error Description	Fix
9	Logic	minutes%students means 0 for length of meetings with test case (50%2=0)	Change to minutes/students to get length of meeting

Error 2

Line Number	Error Type	Error Description	Fix
3	Logic	Will only be satisfied with negative students AND negative minutes but should be satisfied with only one of these for the return message to match and otherwise will return either a negative meeting length or tells the teacher go get a lot of additional time	Change && to

3. divisibleBySeven

/* This method checks if a given number is divisible by 7, returning true if it is prime, false otherwise. */

```
1 public static boolean divisibleBySeven(int number){
2     if (number % 7 != 0) {
3         return true;
4     }
5     return true;
6 }
```

Handwritten note: So when the condition is not satisfied even be satisfied here

Error 1

Line Number	Error Type	Error Description	Fix
2	Syntax	= is used to assign variables and is not a conditional. The program will not compile. We are checking to see equality so we need to use ==	Change = to ==

Error 2

Line Number	Error Type	Error Description	Fix
5	Logic	The else also returns true but the else means that if number/7 does return a remainder (meaning it is not divisible by 7) then it is divisible by 7 which is just a false statement and means that the method will return true no matter what	Change to return false;

4. temperatureConverter

/* This method converts a given temperature in Celsius to the corresponding temperature in Fahrenheit.

E.g. 0.0 degrees Celsius = 32.0 degrees Fahrenheit

E.g. 16.5 degrees Celsius = 61.7 degrees Fahrenheit */

```
1 public static int temperatureConverter(double celsius){  
2     double fahrenheit = (9/5) * celsius + 32;  
3     return fahrenheit;  
4 }
```

*Make
absolute*

Error 1

Line Number	Error Type	Error Description	Fix
1	Syntax	Will return an int for a value that should be a double, narrowing conversion so the program will not compile	Change return type to double

Error 2

Line Number	Error Type	Error Description	Fix
2	Logic	Integer division so 9/5 becomes 1 so it's just Celsius + 32	Rewrite line 2 as double fahrenheit = ((double)9/5) * celsius + 32;

5. checkNumber

/* This method returns a String indicating if the given value is positive, negative, or zero.

*/

```
1 public static String checkNumber(int n) {
```

```
2     if (n > 0) {
```

```
3         return "Positive";
```

```
4     }
```

```
5     if (n != 0) {
```

```
6         return "Zero";
```

```
7     }
```

```
8     if (n < 0); {
```

```
9         return "Negative";
```

```
10 }
```

```
11 }
```

Would mean
that
checkNumber(1.8)
Would give lossy conversion
error

Error 1

Line Number	Error Type	Error Description	Fix
5	Logic	Statement being true depends on n not being zero and if n is not zero then it will be true, so it will return zero. This the opposite of what we want, and results from using n != 0, which means that n will not be 0.	Change if (n!=0){ to if(n==0){ instead

Error 2

Line Number	Error Type	Error Description	Fix
1	Syntax	If we want this to work for any number and not just integers, then right now if we put a non-integer as n we will get a lossy conversion error	Replace int n with double n